Practical No. 7: Implement Random Forest Algorithm and Evaluate the Performance

Submitted By: Shantanu Yadav

Roll No.: 75

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")
```

In [2]:
```
df = pd.read_csv("sales_data.csv")
df
```

Out[2]:

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Produ
0	1052	2023-02-03	Bob	North	5053.97	18	
1	1093	2023-04-21	Bob	West	4384.02	17	
2	1015	2023-09-21	David	South	4631.23	30	
3	1072	2023-08-24	Bob	South	2167.94	39	
4	1061	2023-03-24	Charlie	East	3750.20	13	
...	...	...	...	...	...	...	
995	1010	2023-04-15	Charlie	North	4733.88	4	
996	1067	2023-09-07	Bob	North	4716.36	37	
997	1018	2023-04-27	David	South	7629.70	17	
998	1100	2023-12-20	David	West	1629.47	39	
999	1086	2023-08-16	Alice	East	4923.93	48	

1000 rows × 14 columns

In [3]:
```
df.columns
```

Out[3]:
```
Index(['Product_ID', 'Sale_Date', 'Sales_Rep', 'Region', 'Sales_Amount',
       'Quantity_Sold', 'Product_Category', 'Unit_Cost', 'Unit_Price',
       'Customer_Type', 'Discount', 'Payment_Method', 'Sales_Channel',
       'Region_and_Sales_Rep'],
      dtype='object')
```

In [4]:
```
df.shape
```

Out[4]:
```
(1000, 14)
```

In [5]:
```
df.describe()
```

Out[5]:

	Product_ID	Sales_Amount	Quantity_Sold	Unit_Cost	Unit_Price	Disc
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	1000.0
mean	1050.128000	5019.265230	25.355000	2475.304550	2728.440120	0.1
std	29.573505	2846.790126	14.159006	1417.872546	1419.399839	0.0
min	1001.000000	100.120000	1.000000	60.280000	167.120000	0.0
25%	1024.000000	2550.297500	13.000000	1238.380000	1509.085000	0.0
50%	1051.000000	5019.300000	25.000000	2467.235000	2696.400000	0.1
75%	1075.000000	7507.445000	38.000000	3702.865000	3957.970000	0.2
max	1100.000000	9989.040000	49.000000	4995.300000	5442.150000	0.3

In [6]:
```
df.isnull().sum()
```

Out[6]:
```
Product_ID             0
Sale_Date              0
Sales_Rep              0
Region                 0
Sales_Amount           0
Quantity_Sold          0
Product_Category       0
Unit_Cost              0
Unit_Price             0
Customer_Type          0
Discount               0
Payment_Method         0
Sales_Channel          0
Region_and_Sales_Rep   0
dtype: int64
```

In [10]:
```
df.dtypes
```

Out[10]:
```
Product_ID                int64
Sale_Date                object
Sales_Rep                object
Region                   object
Sales_Amount            float64
Quantity_Sold             int64
Product_Category         object
Unit_Cost               float64
Unit_Price              float64
Customer_Type            object
Discount                float64
Payment_Method           object
Sales_Channel            object
Region_and_Sales_Rep     object
dtype: object
```

In [16]:
```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

df['Sale_Date'] = le.fit_transform(df['Sale_Date'])
df['Sales_Rep'] = le.fit_transform(df['Sales_Rep'])
df['Region'] = le.fit_transform(df['Region'])
df['Product_Category'] = le.fit_transform(df['Product_Category'])
df['Payment_Method'] = le.fit_transform(df['Payment_Method'])
df['Sales_Channel'] = le.fit_transform(df['Sales_Channel'])
df['Region_and_Sales_Rep'] = le.fit_transform(df['Region_and_Sales_Rep'])
```

In [17]:
```
df
```

Out[17]:

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Prodı
0	1052	31	1	1	5053.97	18	
1	1093	102	1	3	4384.02	17	
2	1015	246	3	2	4631.23	30	
3	1072	222	1	2	2167.94	39	
4	1061	76	2	0	3750.20	13	
...	...	...	...	...	...	...	
995	1010	96	2	1	4733.88	4	
996	1067	234	1	1	4716.36	37	
997	1018	108	3	2	7629.70	17	
998	1100	328	3	3	1629.47	39	
999	1086	215	0	0	4923.93	48	

1000 rows × 14 columns

In [28]:
```
X  = df.drop('Sales_Channel',axis = 1)
y = df['Sales_Channel']
```

In [29]:
```
X
```

Out[29]:

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Produ
0	1052	31	1	1	5053.97	18	
1	1093	102	1	3	4384.02	17	
2	1015	246	3	2	4631.23	30	
3	1072	222	1	2	2167.94	39	
4	1061	76	2	0	3750.20	13	
...	...	...	...	...	...	...	
995	1010	96	2	1	4733.88	4	
996	1067	234	1	1	4716.36	37	
997	1018	108	3	2	7629.70	17	
998	1100	328	3	3	1629.47	39	
999	1086	215	0	0	4923.93	48	

1000 rows × 13 columns

In [30]:
```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

In [31]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

In [32]:
```
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42)
```

In [33]:
```
model.fit(X_train, y_train)
```

Out[33]:

▾ RandomForestClassifier

▶ Parameters

In [34]:
```
y_pred = model.predict(X_test)
```

In [39]:
```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)
```

```
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
Accuracy: 0.5

Confusion Matrix:
[[46 55]
 [45 54]]

Classification Report:
              precision    recall  f1-score   support

           0       0.51      0.46      0.48       101
           1       0.50      0.55      0.52        99

    accuracy                           0.50       200
   macro avg       0.50      0.50      0.50       200
weighted avg       0.50      0.50      0.50       200
```